



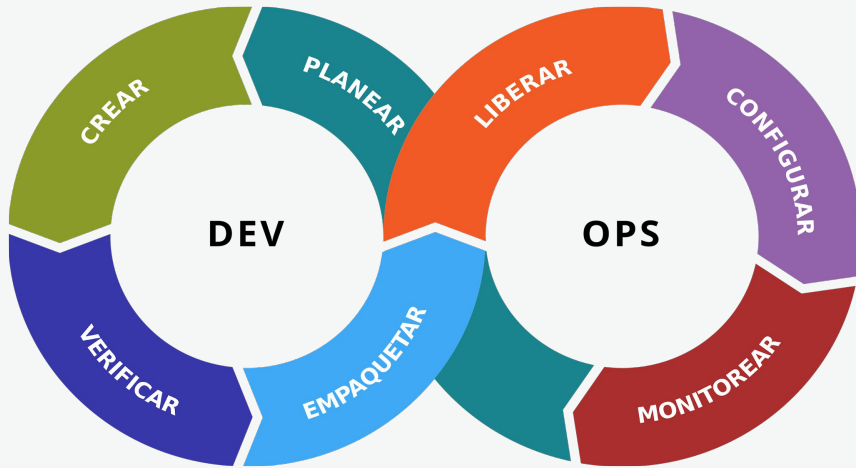
CI/CD Contd.



Topics Covered

- CI/CD
- Deployment Patterns
 - Blue/green
 - Canary
 - Ramped
- Github Action

What is CI/CD?



CI/CD stands for **Continuous Integration and Continuous Delivery**.

- **integration:** build, package, and test applications automatically
- **delivery:** automated deployment of software with little to no downtime for users

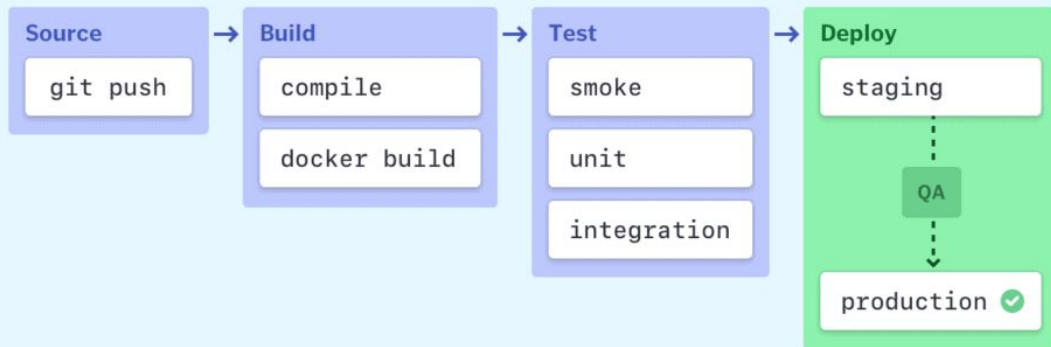
“The continuous integration/continuous delivery (CI/CD) pipeline is an agile DevOps workflow focused on a frequent and reliable software delivery process. The methodology is iterative, rather than linear, which allows DevOps teams to write code, integrate it, run tests, deliver releases and deploy changes to the software collaboratively and in real-time.”

– IBM

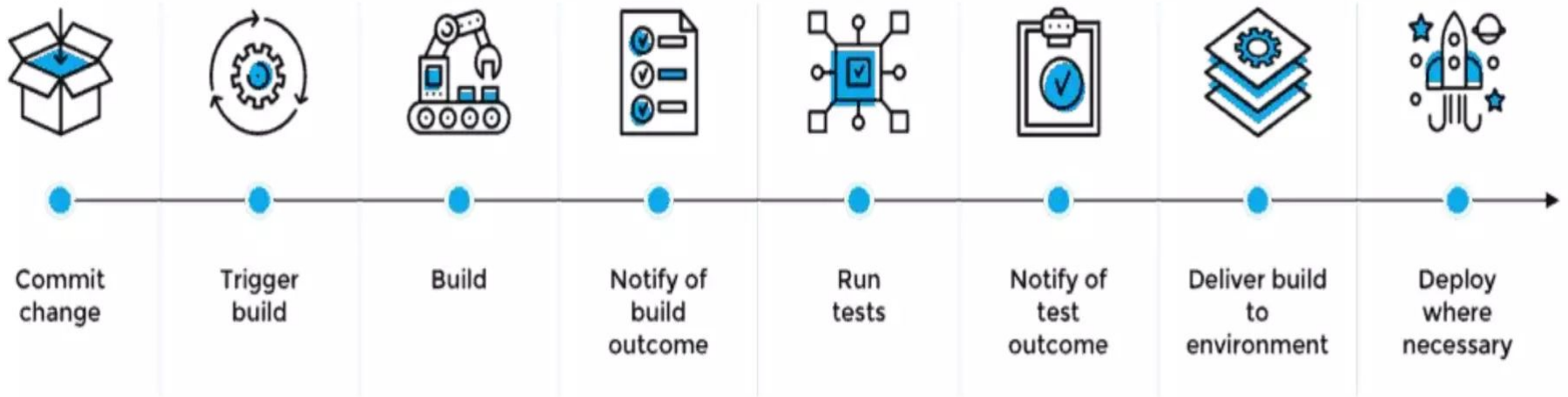
Pipelines

Pipelines describe the automated processes involved in building and deploying code.

By monitoring the status of a pipeline, we can identify issues with the build.



CI/CD pipeline





CI example

Most CI/CD work is using yaml files (e.g. jenkinsfiles, workflows, docker-compose).

This workflow:

- is named “lint”
- is run on pushes to the main and develop branch and when pull requests are created
- steps:
 - sets up node environment
 - checks out the github repo
 - uses cached dependencies (if applicable)
 - uses cached build (if applicable)
 - installs dependencies if need be
 - builds the frontend application into static files
 - runs the linting command

```
1  name: lint
2
3  on:
4    push:
5      branches:
6        - develop
7        - main
8    pull_request:
9
10 jobs:
11   reactci:
12     name: reactci-lint
13     runs-on: ubuntu-latest
14     steps:
15       - name: Set up Node 14.x
16         uses: actions/setup-node@v1
17         with:
18           node-version: 14.x
19
20       - uses: actions/checkout@v2
21
22       - name: Cache dependencies
23         uses: actions/cache@v2
24         with:
25           path: '**/node_modules'
26           key: ${{ runner.os }}-modules-${{ hashFiles('**/yarn.lock') }}
27
28       - name: Cache next build
29         uses: actions/cache@v2
30         with:
31           path: ${{ github.workspace }}/.next/cache
32           key: ${{ runner.os }}-nextjs-${{ hashFiles('**/yarn.lock') }}
33
34       - name: Install dependencies
35         run: yarn --prefer-offline
36
37       - name: Build
38         run: yarn run build
39
40       - name: Run linter
41         run: yarn run lint
```

CI example

add caching of dependencies #33

[Merged](#) jcserv merged 3 commits into `master` from `feature/ci` 6 days ago

Conversation 1 Commits 3 Checks 1 Files changed 2

✓ add next build caching 45b3867

lint
on: pull_request

✓ reactci-lint

reactci-lint
succeeded 6 days ago in 49s

- Set up job
- Set up Node 14.x
- Run actions/checkout@v2
- Cache dependencies
- Cache next build
- Install dependencies
- Build
- Run linter
- Post Cache next build
- Post Cache dependencies
- Post Run actions/checkout@v2
- Complete job

if any jobs fail (ex. the linting step), then that will be visible on the pull request and merging will be prevented.

Add more commits by pushing to the `feature/ui-tweaks` branch on `jcserv/connectu-fe`.

[Show environments](#)

This branch was successfully deployed
1 active deployment

[Ready for review](#)

This pull request is still a work in progress
Draft pull requests cannot be merged.

[Hide all checks](#)

Some checks were not successful
1 failing and 2 successful checks

✗ **lint / reactci-lint (pull_request)** Failing after 1m — reactci-lint [Details](#)

✓ **AccessLint** — Review complete

✓ **Vercel** — Deployment has completed [Details](#)

[Squash and merge](#) You can also [open this in GitHub Desktop](#) or [view command line instructions](#).



Deployment patterns

To successfully launch a new version of the software solution they provide, DevOps teams use deployment patterns. Using these techniques, network traffic in a production environment is transitioned from the old version to the new version based on the firm's specialty. Deployment patterns can influence downtime and operational costs based on the company's specialty.

Deployment patterns

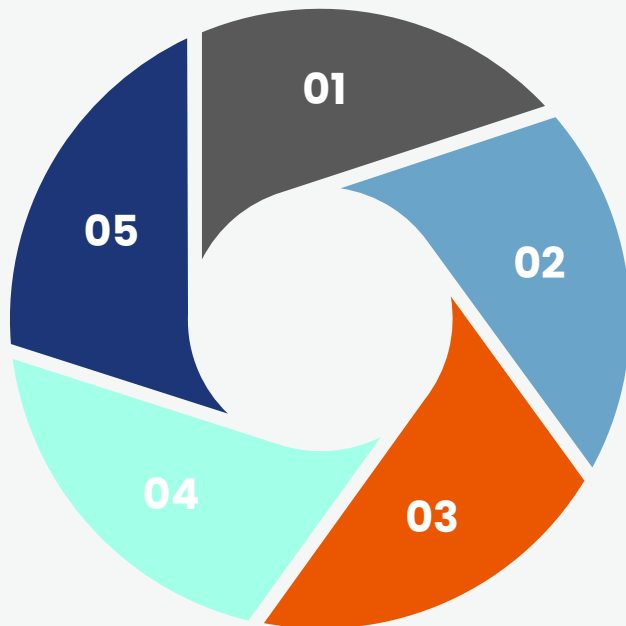
Blue/Green deployment

Canary deployment

Ramped deployment

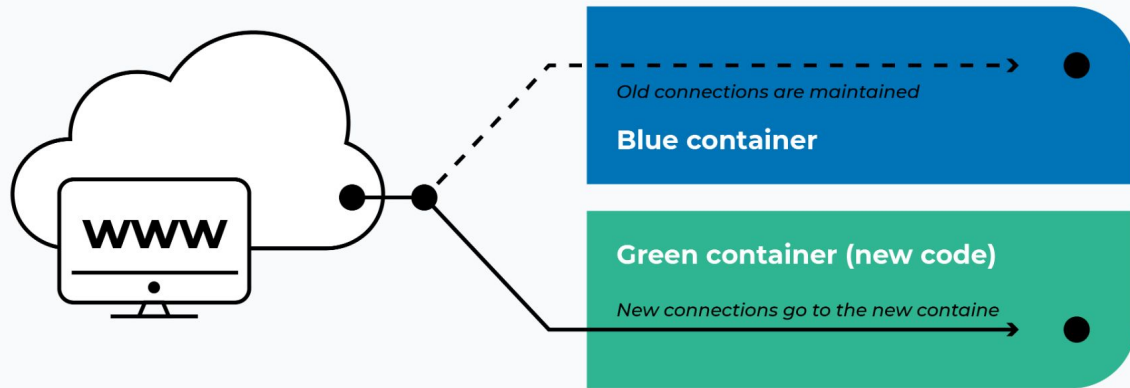
A/B test deployment

Shadow deployment



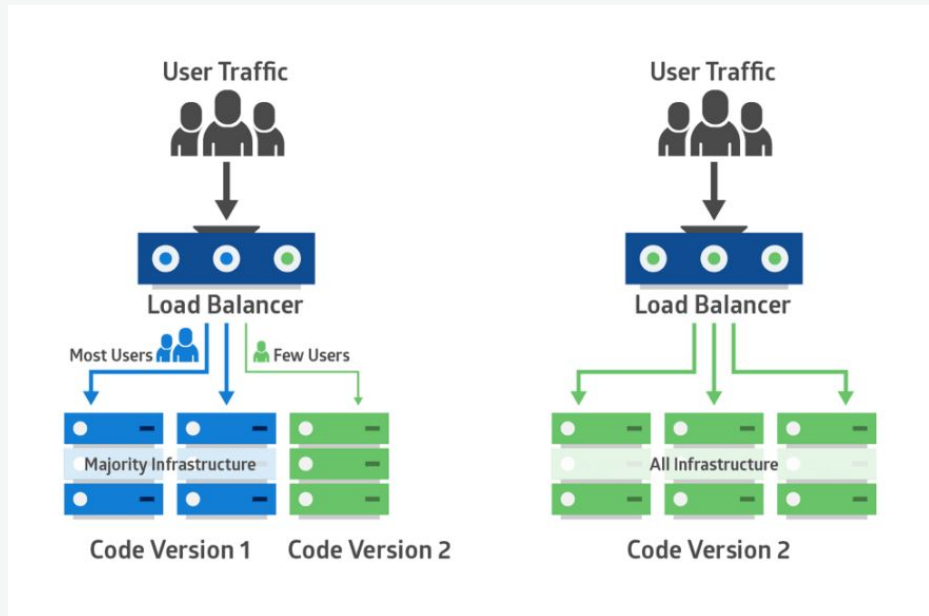
Blue/green deployment

Blue/green deployment is a deployment scheme enabling changes to be pushed while having no downtime for users. Traffic is routed to a container with the old code, while a new container is spun up with the new code. After building, the traffic is rerouted and the old container can be torn down.



Canary deployment

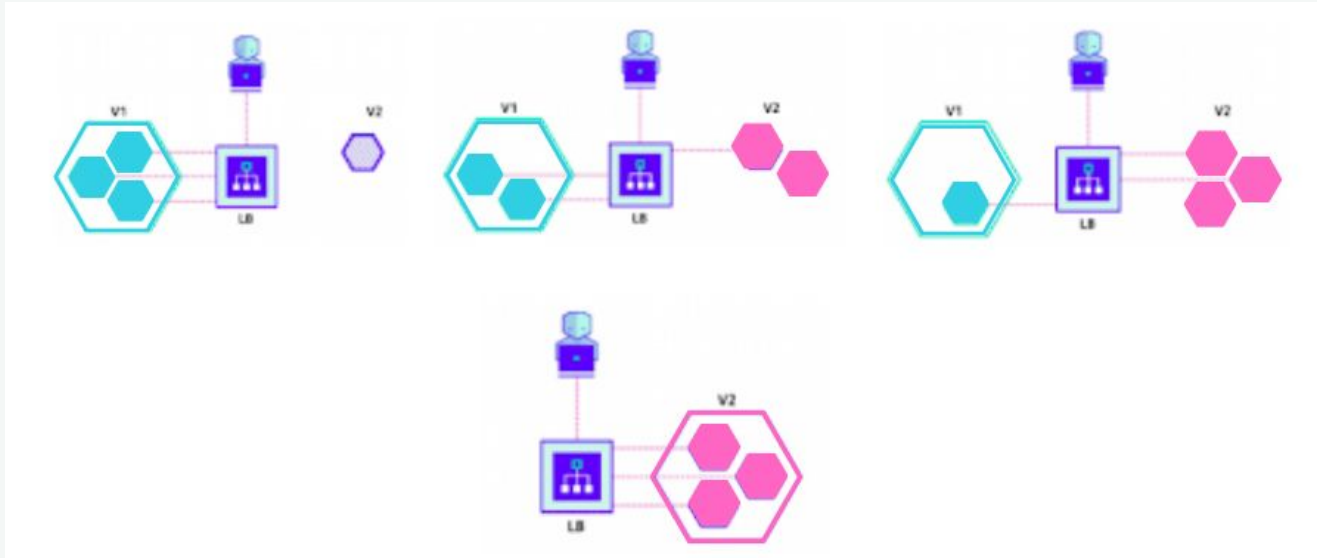
canary deployment is the practice of staging releases to release changes to a small batch of users, receive & address feedback, and then roll out changes to the entire user base gradually.



In coal mine, during the 1900s, miners would take canaries down into the tunnels with them. If dangerous gases such as carbon monoxide collected in the mine, the gases would kill the canary before killing the miners, thus providing a warning to exit the tunnels immediately.

Ramped deployment

Ramped deployment gradually changes the older version to the new version. Unlike canary deployment, the ramped deployment strategy replaces instances of the old application version with instances of the new application version one instance at a time



Why CI/CD?

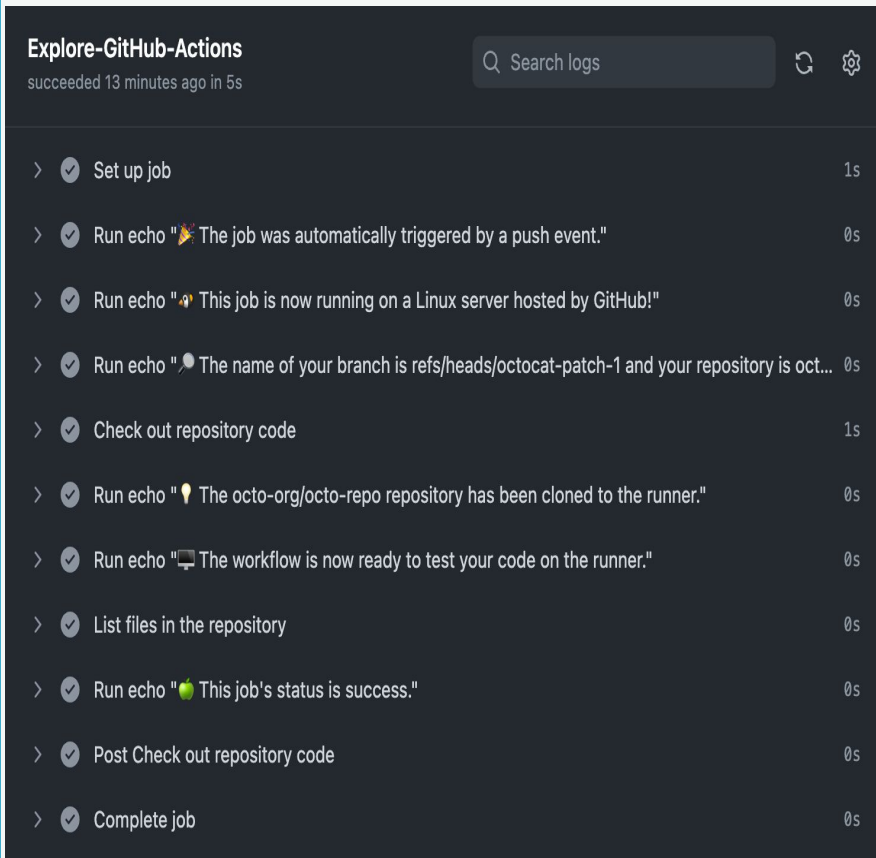
- ✓ **Automation of the test, build & deployment process:** making the development process more efficient, reducing the length of the software delivery process, allowing a developer's changes to a cloud application to go live within minutes of writing them.
- ✓ **Continuous feedback:** Every time code is tested, developers can quickly take action on the feedback and improve the code, promoting agile & test driven development.
- ✓ **Early fault detection:** In continuous integration, testing is automated for each version of code built to look for issues integration. These issues are easier to fix the earlier in the pipeline that they occur.
- ✓ **Improving team collaboration and system integration.** Everyone on the team can change code, respond to feedback and quickly respond to any issues that occur.



GitHub Action

GitHub Actions is a continuous integration and continuous delivery (CI/CD) platform that allows you to automate your build, test, and deployment pipeline.

You can create workflows that build and test every pull request to your repository, or deploy merged pull requests to production.



Demo Time!

1

Clone/fork the [demo repo](#) from github classroom

2

Complete the workflow

3

Go to your repo on GitHub website and play with the github actions

CI demo

For every push or pr to the main branch, it will run the workflow:

1. Sets up node environment
2. Checks out the github repo
3. Installs dependencies and test for frontend
4. Installs dependencies and test for frontend
5. Sends test coverage to some code analysis platform (optional)

```
ub > workflows > .github\actions-workflow.yml
name: CI/CD
run-name: ${{ github.actor }} is running ${{ github.workflow }} on ${{ github.ref }}
on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    strategy:
      matrix:
        node-version: [12.x]
    steps:
      - name: Clone repository
        uses: actions/checkout@v2

      - name: Use Node.js 12.x
        uses: actions/setup-node@v1
        with:
          node-version: 12.x

      - name: Testing frontend
        run: |
          ls
          cd frontend
          npm install
          npm test

      - name: Testing backend
        run: |
          cd backend
          npm install
          npm test

      # - name: Publish code coverage
```

CD demo

1. Checks out the github repo
2. Builds the images with docker-compose
3. Testing (if applicable)
4. Login to Docker Hub with secretes
5. Publish the Docker images to your repositories on Docker Hub

You can then pull and run the published images on any deploy machines!

code

Blame

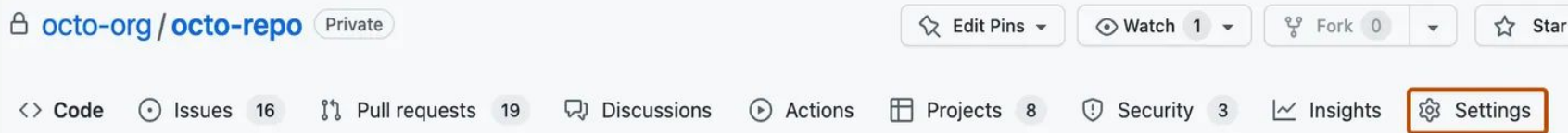
26 lines (23 loc) · 919 Bytes

```
1  name: Build and deploy to Docker Hub
2  on:
3    workflow_dispatch:
4  jobs:
5    build:
6      runs-on: ubuntu-latest
7      timeout-minutes: 30
8      steps:
9        - uses: actions/checkout@v4
10
11        - name: Docker compose build
12          run: docker-compose build
13
14        - name: Log in to Docker Hub
15          uses: docker/login-action@f4ef78c080cd8ba55a85445d5b36e214a81df20a
16          with:
17            username: ${ secrets.DOCKER_USERNAME }
18            password: ${ secrets.DOCKER_PASSWORD }
19
20        - name: Push to Docker Hub
21          run: |
22            docker tag ci-cd-example_frontend:latest ${ secrets.DOCKER_USERNAME }/cicd-demo-frontend:latest
23            docker tag ci-cd-example_backend:latest ${ secrets.DOCKER_USERNAME }/cicd-demo-backend:latest
24            docker push ${ secrets.DOCKER_USERNAME }/cicd-demo-frontend:latest
25            docker push ${ secrets.DOCKER_USERNAME }/cicd-demo-backend:latest
26            docker images
```

GitHub Secret

Development environment secrets allow you to store sensitive information in your organization or repository for use with GitHub Codespaces.

You should not reveal any password or API keys in a public (or even private) repo!!!



Actions secrets and variables

Secrets and variables allow you to manage reusable configuration data. Secrets are **encrypted** and are used for sensitive data. [Learn more about encrypted secrets](#). Variables are shown as plain text and are used for **non-sensitive** data. [Learn more about variables](#).

Anyone with collaborator access to the repositories with access to a secret or variable can use it for Actions. They are not passed to workflows that are triggered by a pull request from a fork.



Using Secrets in Github Actions

To provide an action with a secret as an input or environment variable, you can use the secrets context to access secrets you've created in your repository.

```
steps:
  - name: Hello world action
    with: # Set the secret as an input
      super_secret: ${ secrets.SuperSecret }
    env: # Or as an environment variable
      super_secret: ${ secrets.SuperSecret }
```

DockerHub

Docker Hub is a container registry built for developers and open source contributors to find, use, and share their container images.

[Repositories](#) / [Create](#) Using 0 of 1 private repositories.

Create repository

Namespace
shang8024

Repository Name *



Short description

A short description to identify your repository. If the repository is public, this description is used to index your content on Docker Hub and in search engines, and is visible to users in search results.

Visibility

Using 0 of 1 private repositories. [Get more](#)

☒ **Public** 
Appears in Docker Hub search results

☐ **Private** 
Only visible to you

Cancel

Create

Pushing images

You can push a new image to this repository using the CLI:

```
docker tag local-image:tagname new-repo:tagname
docker push new-repo:tagname
```

Make sure to replace `tagname` with your desired image repository tag.

[Settings](#) / [Personal access tokens](#)

Personal access tokens

You can use a personal access token instead of a password for Docker CLI authentication. Create multiple tokens, control their scope, and delete tokens at any time. [Learn more](#)

Generate new token

Description	Scope	Status	Source	Created	Last Used	
Generated through desktop a...	Read, Write, Delete	Active	Auto-generated	Jul 22, 2024 at 14:41:05	Jul 22, 2024 at 17:36:47	⋮
GitAction	Read & Write	Active	Manual	Jul 22, 2024 at 15:34:59	Jul 22, 2024 at 17:00:37	⋮
Generated through desktop a...	Read, Write, Delete	Active	Auto-generated	Jul 04, 2022 at 12:43:36	Jul 04, 2022 at 13:02:41	⋮
github	Read, Write, Delete	Active	Manual	Jul 23, 2024 at 13:21:54	Never	⋮
Azure	Read, Write, Delete	Active	Manual	Jul 22, 2024 at 17:00:34	Never	⋮
Generated through desktop a...	Read, Write, Delete	Active	Auto-generated	Mar 25, 2024 at 19:43:39	Never	⋮

Rows per page: 10 1-6 of 6

You will need to login to the dockerhub to publish the image on your login machine, or use a personal access token for CLI authentication.

Create Web App ...

Instance details

Name *

☒ Try a unique default hostname (preview). [More about unique hostnames](#)

Publish * ☐ Code ☒ Container ☐ Static Web App

Operating System * ☒ Linux ☐ Windows

Region *

 Not finding your App Service Plan? Try a different region or a Service Environment.

Pricing plans

App Service plan pricing tier determines the location, features, cost and compute resources associated with the plan. [Learn more](#)

Linux Plan (East US) * 
[Create new](#)

Pricing plan **Free F1** (Shared infrastructure)

Zone redundancy

An App Service plan can be deployed as a zone redundant service in the regions that support it. This is a time only decision. You can't make an App Service plan zone redundant after it has been deployed. [Learn more](#)

Zone redundancy ☐ **Enabled:** Your App Service plan and the apps in it will be zone redundant. The minimum App Service plan instance count is 2.

[Review + create](#)

[< Previous](#)

[Next : Container >](#)

We will be using Azure for deployment this time!

1. Go to Azure web service website, sign up for a free account
2. Click on "Create a Resource" and create a "web app"
3. For instance Details, choose "Container" as Publish
4. For pricing plan, create a new one and choose free plan
5. Next+Container, choose "Docker hub" → "docker compose" → "Public"
6. Fill out the remaining required field whatever you like, and create your web app!

Create Web App ...

Basics Container Networking Monitor + secure Tags Review + create

Select your preferred source for container images. You can change these settings and other dependencies the app. [Learn more](#)

Sidecar support (preview) ⓘ

- ☐ Enabled
☒ Disabled

Image Source *

- ☐ Quickstart
☐ Azure Container Registry
☒ Docker Hub or other registries

Options

- ☐ Single Container
☒ Docker Compose (Preview)

Docker hub options

Access Type *

- ☒ Public
☐ Private

Configuration File

Select a file

Configuration ⓘ

Review + create

< Previous

Next : Networking >

We will be using Azure for deployment this time!

1. Go to Azure web servie website, signup for a free account
2. Click on "Create Resource" and create a "web app"
3. For Instance Details, choose "Container" as Publish
4. For Pricing plan, create a new one and choose free plan
5. Next+Container, choose "Docker hub" → "docker compose" → "Public"
6. Fill out the remaining required field whatever you like, and create your web app!
7. After creation completed, click on "go to resource" and then go to deployment center

Create Web App ...

Basics Container Networking Monitor + secure Tags Review + create

Select your preferred source for container images. You can change these settings and other dependencies the app. [Learn more](#)

Sidecar support (preview) ⓘ

- ☐ Enabled
☒ Disabled

Image Source *

- ☐ Quickstart
☐ Azure Container Registry
☒ Docker Hub or other registries

Options

- ☐ Single Container
☒ Docker Compose (Preview)

Docker hub options

Access Type *

- ☒ Public
☐ Private

Configuration File

Select a file

Configuration ⓘ

Review + create

< Previous

Next : Networking >

We will be using Azure for deployment this time!

1. Go to Azure web servie website, signup for a free account
2. Click on “Create Resource” and create a “web app”
3. For Instance Details, choose “Container” as Publish
4. For Pricing plan, create a new one and choose free plan
5. Next+Container, choose “Docker hub” → “docker compose” → “Public”
6. Fill out the remaining required field whatever you like, and create your web app!
7. After creation completed, click on “go to resource” and then go to deployment center